INFORME TÉCNICO: TALLER 7 (WORKSHOP 7)

Intercambio de Datos Estructurados usando JSON y XML en Sistemas Distribuidos

Materia: Sistemas Distribuidos
Semestre: Semestre II - 2026
Profesor: Prof. Francisco Hidrobo
Fecha: Noviembre 2026
Proyecto: *AIR_QUALITY_DISTRIBUTED* - Sistema Distribuido y Tolerante a Fallos para Aprendizaje Federado sobre Calidad del Aire

Resumen Ejecutivo

En este taller se diseñó, implementó y evaluó el intercambio estructurado de información entre componentes distribuidos ejecutados en nodos independientes de la arquitectura del proyecto. Se desarrollaron dos actividades principales:

1. **Actividad 1 (Servicio con Esquemas Predefinidos):** Se implementó el servicio `get_global_model` para la sincronización de pesos y parámetros entre el **Nodo 0 (Servidor Coordinador)** y los **Nodos Clientes (Nodo 1: PC / Nodo 2: Raspberry Pi)**, validando tanto la solicitud como la respuesta contra esquemas formales predefinidos (**JSON Schema** [Draft-07] y **XML Schema Definition - XSD**). Se incorporaron casos de control negativo para verificar la capacidad del esquema para rechazar payloads malformados o no autorizados.
2. **Actividad 2 (Servicio sin Esquema Predefinido):** Se implementó el servicio `training_report` para la notificación de métricas locales de entrenamiento (número de muestras procesadas, función de pérdida local y ronda), aplicando validación programática semántica directamente en el código de la aplicación.
3. **Evaluación Experimental:** Se instrumentaron mediciones precisas sobre el canal TCP midiendo: (a) cantidad de datos transferidos en bytes, (b) latencia de ida y vuelta de la red (*Round Trip Time - RTT*), y (c) sobrecarga de procesamiento desagregada en serialización, validación y deserialización.

1. Arquitectura Distribuida y Selección de Componentes

La arquitectura del proyecto está compuesta por tres nodos físicos interconectados mediante una red de área local (LAN) o enlace Wi-Fi privado:

```
<div style="text-align: center; margin: 20px 0; page-break-inside: avoid;">
  <svg width="780" height="340" viewBox="0 0 780 340" xmlns="http://www.w3.org/2000/svg" style="max-width: 100%; font-family: 'Segoe UI', Rol
    <defs>
      <linearGradient id="gradServer" x1="0%" y1="0%" x2="100%" y2="100%">
        <stop offset="0%" stop-color="#1e3a8a"/>
        <stop offset="100%" stop-color="#2563eb"/>
      </linearGradient>
      <linearGradient id="gradClient1" x1="0%" y1="0%" x2="100%" y2="100%">
        <stop offset="0%" stop-color="#065f46"/>
        <stop offset="100%" stop-color="#059669"/>
      </linearGradient>
      <linearGradient id="gradClient2" x1="0%" y1="0%" x2="100%" y2="100%">
        <stop offset="0%" stop-color="#831843"/>
        <stop offset="100%" stop-color="#be185d"/>
      </linearGradient>
      <filter id="shadow" x="-5%" y="-5%" width="110%" height="115%">
        <feDropShadow dx="0" dy="3" stdDeviation="4" flood-color="#000" flood-opacity="0.12"/>
      </filter>
      <marker id="arrowhead" markerWidth="8" markerHeight="6" refX="7" refY="3" orient="auto">
        <polygon points="0 0, 8 3, 0 6" fill="#475569"/>
      </marker>
      <marker id="arrowhead-start" markerWidth="8" markerHeight="6" refX="1" refY="3" orient="auto">
        <polygon points="8 0, 0 3, 8 6" fill="#475569"/>
      </marker>
    </defs>

    <!-- Fondo del canvas -->
    <rect width="780" height="340" rx="12" fill="#f8fafc" stroke="#e2e8f0" stroke-width="1.5"/>

    <!-- NODO 1: CLIENTE 1 (PC 2) -->
    <g filter="url(#shadow)">
      <rect x="30" y="30" width="230" height="125" rx="8" fill="#ffffff" stroke="#059669" stroke-width="2"/>
      <rect x="30" y="30" width="230" height="32" rx="8" fill="url(#gradClient1)"/>
      <rect x="30" y="54" width="230" height="8" fill="url(#gradClient1)"/>
      <text x="145" y="52" fill="#ffffff" font-size="12" font-weight="bold" text-anchor="middle">Nodo 1: Cliente 1 (PC)</text>
      <text x="45" y="80" fill="#1e293b" font-size="10.5" font-weight="600">• TCP Client Socket</text>
```

```
    <text x="45" y="98" fill="#1e293b" font-size="10.5">• Validador Esquemas / Código</text>
    <text x="45" y="116" fill="#1e293b" font-size="10.5">• Datos Locales (Partición 1)</text>
    <text x="45" y="134" fill="#64748b" font-size="9.5">IP: 192.168.1.101 | Windows/Linux</text>
  </g>

  <!— NODO 2: CLIENTE 2 (Raspberry Pi Edge) —>
  <g filter="url(#shadow)">
    <rect x="30" y="185" width="230" height="125" rx="8" fill="#ffffff" stroke="#be185d" stroke-width="2"/>
    <rect x="30" y="185" width="230" height="32" rx="8" fill="url(#gradClient2)"/>
    <rect x="30" y="209" width="230" height="8" fill="url(#gradClient2)"/>
    <text x="145" y="207" fill="#ffffff" font-size="12" font-weight="bold" text-anchor="middle">Nodo 2: Cliente 2 (Raspberry Pi)</text>
    <text x="45" y="235" fill="#1e293b" font-size="10.5" font-weight="600">• TCP Client Socket</text>
    <text x="45" y="253" fill="#1e293b" font-size="10.5">• Validador Esquemas / Código</text>
    <text x="45" y="271" fill="#1e293b" font-size="10.5">• Nodo Edge Recursos Restringidos</text>
    <text x="45" y="289" fill="#64748b" font-size="9.5">IP: 192.168.1.102 | Raspberry Pi OS</text>
  </g>

  <!— NODO 0: SERVIDOR COORDINADOR (PC 1) —>
  <g filter="url(#shadow)">
    <rect x="490" y="45" width="260" height="250" rx="8" fill="#ffffff" stroke="#2563eb" stroke-width="2"/>
    <rect x="490" y="45" width="260" height="34" rx="8" fill="url(#gradServer)"/>
    <rect x="490" y="71" width="260" height="8" fill="url(#gradServer)"/>
    <text x="620" y="68" fill="#ffffff" font-size="13" font-weight="bold" text-anchor="middle">Nodo 0: Coordinador / Servidor</text>

    <rect x="505" y="90" width="230" height="32" rx="4" fill="#eff6ff" stroke="#bfdbfe"/>
    <text x="620" y="110" fill="#1e3a8a" font-size="11" font-weight="bold" text-anchor="middle">TCP Server (0.0.0.0:5000)</text>

    <rect x="505" y="130" width="230" height="48" rx="4" fill="#f8fafc" stroke="#cbd5e1"/>
    <text x="515" y="148" fill="#0f172a" font-size="10" font-weight="bold">Validación de Esquemas:</text>
    <text x="515" y="165" fill="#475569" font-size="9.5">JSON Schema (Draft-07) &amp;amp; XSD (lxml)</text>

    <rect x="505" y="186" width="230" height="48" rx="4" fill="#f8fafc" stroke="#cbd5e1"/>
    <text x="515" y="204" fill="#0f172a" font-size="10" font-weight="bold">Despacho de Servicios:</text>
    <text x="515" y="221" fill="#475569" font-size="9.5">1. get_global_model | 2. training_report</text>

    <rect x="505" y="242" width="230" height="42" rx="4" fill="#f8fafc" stroke="#cbd5e1"/>
    <text x="515" y="259" fill="#0f172a" font-size="10" font-weight="bold">Agregación &amp;amp; Checkpoints:</text>
    <text x="515" y="274" fill="#475569" font-size="9">FedAvg, Idempotencia UUID, Logs</text>
  </g>

  <!— ENLACES DE COMUNICACIÓN (FLECHAS BIDIRECCIONALES) —>
  <!— Línea Cliente 1 <—> Servidor —>
  <path d="M 260 92 L 490 120" stroke="#475569" stroke-width="2.5" stroke-dasharray="6,4" marker-end="url(#arrowhead)" marker-start="url(#a
  <!— Etiqueta Enlace 1 —>
  <rect x="290" y="76" width="170" height="26" rx="4" fill="#ffffff" stroke="#94a3b8" stroke-width="1"/>
  <text x="375" y="93" fill="#0f172a" font-size="9.5" font-weight="600" text-anchor="middle">TCP / JSON &amp;amp; XML Framing</text>

  <!— Línea Cliente 2 <—> Servidor —>
  <path d="M 260 248 L 490 220" stroke="#475569" stroke-width="2.5" stroke-dasharray="6,4" marker-end="url(#arrowhead)" marker-start="url(#
  <!— Etiqueta Enlace 2 —>
  <rect x="290" y="238" width="170" height="26" rx="4" fill="#ffffff" stroke="#94a3b8" stroke-width="1"/>
  <text x="375" y="255" fill="#0f172a" font-size="9.5" font-weight="600" text-anchor="middle">TCP / JSON &amp;amp; XML Framing</text>
 </svg>
</div>
```

Componentes Seleccionados para el Taller:

- **Componente A - Servidor Coordinador (Nodo 0):** Ejecutado en la máquina anfitriona (PC 1), actúa como el proveedor central de servicios de sincronización y registro. Escucha en un puerto TCP configurable (por defecto `5000`) y expone los servicios `get_global_model` y `training_report`.
- **Componente B - Cliente Federado (Nodo 1 / Nodo 2):** Ejecutado en una máquina remota (PC 2 o Raspberry Pi), consume los servicios del coordinador para solicitar la versión actual del modelo y reportar el estado de su entrenamiento local.

Delimitación de Mensajes sobre TCP (*Message Framing*):

Dado que TCP es un protocolo orientado a flujo de bytes (*stream-oriented*) sin delimitación intrínseca de mensajes, se implementó una capa de delimitación con prefijo binario de longitud (*Length-Prefix Framing*):

- Cada mensaje va precedido por un encabezado de 4 bytes en formato Big-Endian (`>I`), indicando con exactitud la longitud del payload XML o JSON.
- Esto previene cualquier condición de fragmentación de paquetes (*TCP segmentation*) o concatenación indebida (*sticky packets*).

---

2. Actividad 1: Servicio con Esquemas Predefinidos (`get_global_model`)

2.1. Descripción del Servicio

El servicio `get_global_model` es el punto de entrada crítico de cada ronda federada. El cliente notifica su identificador, el servicio requerido, la ronda en curso y el formato deseado (`json` o `xml`). El Servidor valida la solicitud, extrae el estado actual del modelo y responde con la confirmación, versión de modelo y mensaje descriptivo.

2.2. Esquema Formal de Solicitud (Request Schema)

JSON Schema (`schemas/request.schema.json`):

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "RequestSchema",
  "type": "object",
  "properties": {
    "client_id": {
      "type": "string",
      "enum": ["client_1", "client_2"]
    },
    "service": {
      "type": "string",
      "enum": ["get_global_model", "ping", "disconnect"]
    },
    "round": {
      "type": "integer",
      "minimum": 0
    },
    "format": {
      "type": "string",
      "enum": ["json", "xml"]
    }
  },
  "required": ["client_id", "service", "round", "format"],
  "additionalProperties": true
}
```

XML Schema Definition (`schemas/message.xsd` - Fragmento de Solicitud):

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema" elementFormDefault="qualified">

  <xs:element name="request" type="RequestType"/>

  <xs:complexType name="RequestType">
    <xs:sequence>
      <xs:element name="client_id">
        <xs:simpleType>
          <xs:restriction base="xs:string">
            <xs:enumeration value="client_1"/>
            <xs:enumeration value="client_2"/>
          </xs:restriction>
        </xs:simpleType>
      </xs:element>
      <xs:element name="service">
        <xs:simpleType>
          <xs:restriction base="xs:string">
            <xs:enumeration value="get_global_model"/>
            <xs:enumeration value="ping"/>
            <xs:enumeration value="disconnect"/>
          </xs:restriction>
        </xs:simpleType>
      </xs:element>
      <xs:element name="round" type="xs:nonNegativeInteger"/>
      <xs:element name="format">
        <xs:simpleType>
          <xs:restriction base="xs:string">
            <xs:enumeration value="json"/>
            <xs:enumeration value="xml"/>
          </xs:restriction>
        </xs:simpleType>
      </xs:element>
    </xs:sequence>
  </xs:complexType>
</xs:schema>
```

2.3. Esquema Formal de Respuesta (Response Schema)

JSON Schema (`schemas/response.schema.json`):

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "ResponseSchema",
  "type": "object",
  "properties": {
    "status": {
      "type": "string",
      "enum": ["success", "received", "error"]
    },
    "round": {
      "type": "integer",
      "minimum": 0
    },
    "model_version": {
      "type": "integer",
      "minimum": 0
    },
    "message": {
      "type": "string"
    },
    "client_id": {
      "type": "string"
    }
  },
  "required": ["status", "round", "message"],
  "additionalProperties": true
}
```

XML Schema Definition (`schemas/message.xsd` - Fragmento de Respuesta):

```
<xs:element name="response" type="ResponseType"/>

<xs:complexType name="ResponseType">
  <xs:sequence>
    <xs:element name="status">
      <xs:simpleType>
        <xs:restriction base="xs:string">
          <xs:enumeration value="success"/>
          <xs:enumeration value="received"/>
          <xs:enumeration value="error"/>
        </xs:restriction>
      </xs:simpleType>
    </xs:element>
    <xs:element name="round" type="xs:nonNegativeInteger"/>
    <xs:element name="model_version" type="xs:nonNegativeInteger" minOccurs="0"/>
    <xs:element name="message" type="xs:string"/>
    <xs:element name="client_id" type="xs:string" minOccurs="0"/>
  </xs:sequence>
</xs:complexType>
```

2.4. Código de Validación (Cliente y Servidor)

Funciones de Validación (`protocol.py`):

```python
def validate_json_request(data: Dict[str, Any]) -> Tuple[bool, Optional[str]]:
    """Valida la solicitud JSON contra schemas/request.schema.json."""
    try:
        jsonschema.validate(instance=data, schema=_REQUEST_JSON_SCHEMA)
        return True, None
    except jsonschema.ValidationError as e:
        return False, e.message

def validate_json_response(data: Dict[str, Any]) -> Tuple[bool, Optional[str]]:
    """Valida la respuesta JSON contra schemas/response.schema.json."""
    try:
        jsonschema.validate(instance=data, schema=_RESPONSE_JSON_SCHEMA)
        return True, None
    except jsonschema.ValidationError as e:
        return False, e.message

def validate_xml_message(xml_bytes: bytes) -> Tuple[bool, Optional[str]]:
    """Valida mensajes XML (request o response) contra schemas/message.xsd usando lxml."""
    try:
        doc = etree.fromstring(xml_bytes)
        _XML_SCHEMA_VALIDATOR.assertValid(doc)
        return True, None
    except etree.DocumentInvalid as e:
        return False, str(e)
    except Exception as e:
        return False, f"Error de sintaxis XML: {str(e)}"
```

3. Actividad 2: Servicio sin Esquema Predefinido (`training_report`)

3.1. Descripción del Servicio

El servicio `training_report` permite que los clientes notifiquen al servidor que han culminado las épocas locales de entrenamiento. A diferencia de la Actividad 1, **no se utiliza un archivo de esquema externo** (como JSON Schema o XSD). En su lugar, el contrato de datos se valida mediante **reglas de lógica de negocio programadas directamente en código Python**, tanto en el emisor antes de enviar como en el receptor antes de procesar.

3.2. Reglas Semánticas en Código

1. `client_id` : Debe ser de tipo string, no nulo ni vacío, y restringido al conjunto de clientes autorizados (`client_1` , `client_2`).
2. `round` : Debe ser un número entero no negativo ($round \ge 0$).
3. `samples` : Debe ser un número entero estrictamente positivo ($samples > 0$).
4. `local_loss` : Debe ser un valor numérico real no negativo y finito ($loss \ge 0$, descartando valores `NaN` o infinitos).

3.3. Implementación del Validador Programático (`protocol.py`):

```python
def validate_training_report_programmatic(data: Dict[str, Any]) -> Tuple[bool, Optional[str]]:
    """
    Validación semántica en código de aplicación sin esquemas externos.
    """
    client_id = data.get("client_id")
    if not client_id or not isinstance(client_id, str) or client_id not in ["client_1", "client_2"]:
        return False, f"client_id inválido o vacío: '{client_id}'. Debe ser 'client_1' o 'client_2'."

    round_num = data.get("round")
    if round_num is None or not isinstance(round_num, int) or round_num < 0:
        return False, f"round debe ser un entero >= 0, recibido: {round_num}"

    samples = data.get("samples")
    if samples is None or not isinstance(samples, int) or samples <= 0:
        return False, f"samples debe ser un entero > 0, recibido: {samples}"

    local_loss = data.get("local_loss")
    if local_loss is None or not isinstance(local_loss, (int, float)) or local_loss < 0 or local_loss != local_loss:
        return False, f"local_loss debe ser numérico finito >= 0, recibido: {local_loss}"

    return True, None
```

4. Resultados Experimentales y Mediciones

Las pruebas se ejecutaron mediante la suite automatizada del taller (`workshop7_server.py` y `workshop7_client.py`). A continuación se detallan las evidencias capturadas en consola y las mediciones cuantitativas.

4.1. Registro de Ejecución del Cliente (Terminal / Consola)

```
TERMINAL OUTPUT
================================================================================
CLIENTE DISTRIBUIDO – WORKSHOP 7: INTERCAMBIO JSON Y XML
Conectando a Servidor Coordinador en: 127.0.0.1:5990
Identificador del Cliente: client_1
================================================================================

################################################################################
ACTIVIDAD 1: Servicio CON Esquemas Predefinidos ('get_global_model')
Esquemas: schemas/request.schema.json, response.schema.json, message.xsd
################################################################################

[Paso 1.1] Transmisión en formato JSON:
  * Solicitud enviada: {'client_id': 'client_1', 'service': 'get_global_model', 'round': 1, 'format': 'json'}
  * Validación previa solicitud (JSON Schema): VÁLIDA (PASS)
  * Respuesta recibida: {'status': 'success', 'round': 1, 'model_version': 1, 'message': 'Modelo global ronda 1 entregado con éxito a client_1',
  * Validación respuesta (Response Schema): VÁLIDA (PASS)
  * Bytes: Solicitud=86 B | Respuesta=146 B | Total=232 B
  * Latencia RTT: 4.339 ms
  * Sobrecarga procesamiento: Ser=0.031 ms | ValReq=0.638 ms | Deser=0.021 ms | ValResp=0.536 ms | Total=1.226 ms

[Paso 1.2] Transmisión en formato XML:
  * Solicitud enviada: {'client_id': 'client_1', 'service': 'get_global_model', 'round': 1, 'format': 'xml'}
  * Validación previa solicitud (XML XSD): VÁLIDA (PASS)
  * Respuesta recibida: {'status': 'success', 'round': 1, 'model_version': 1, 'message': 'Modelo global ronda 1 entregado con éxito a client_1',
  * Validación respuesta (XML XSD): VÁLIDA (PASS)
  * Bytes: Solicitud=160 B | Respuesta=235 B | Total=395 B
  * Latencia RTT: 0.861 ms
  * Sobrecarga procesamiento: Ser=0.123 ms | ValReq=0.074 ms | Deser=0.096 ms | ValResp=0.414 ms | Total=0.708 ms

[Paso 1.3] Caso de Control Negativo (Payload corrupto violando esquema):
  * Enviando solicitud deliberadamente inválida: {'client_id': 'nodo_no_autorizado_99', 'service': 'get_global_model', 'round': -5, 'format': 'j
  * Detección local por JSON Schema: DETECTADO (OK): -5 is less than the minimum of 0
  * Respuesta del servidor ante payload corrupto: {'status': 'error', 'round': -5, 'message': 'Esquema inválido: -5 is less than the minimum of
  → Rechazo verificado correctamente por el Servidor.

################################################################################
ACTIVIDAD 2: Servicio SIN Esquemas Predefinidos ('training_report')
Validación: Reglas semánticas programáticas en código de aplicación
################################################################################

[Paso 2.1] Reporte de entrenamiento válido:
  * Solicitud enviada: {'client_id': 'client_1', 'service': 'training_report', 'round': 1, 'samples': 3450, 'local_loss': 0.0385, 'format': 'jso
  * Validación en código (Reglas semánticas): APROBADA (PASS)
  * Respuesta recibida del servidor: {'status': 'received', 'round': 1, 'message': 'Reporte de entrenamiento de client_1 (Ronda 1) persistido co
  * Bytes: Solicitud=124 B | Respuesta=145 B | Total=269 B
  * Latencia RTT: 0.379 ms
  * Sobrecarga procesamiento: Ser=0.015 ms | ValCódigo=0.005 ms | Deser=0.012 ms | Total=0.033 ms

[Paso 2.2] Caso de Control Negativo (Regla semántica violada: samples ≤ 0):
  * Enviando reporte con muestras negativas: {'client_id': 'client_1', 'service': 'training_report', 'round': 1, 'samples': -450, 'local_loss':
  * Detección por validador programático: DETECTADO (OK): samples debe ser un entero > 0, valor recibido: -450
  * Respuesta del servidor: {'status': 'error', 'round': 1, 'message': 'Validación programática falló: samples debe ser un entero > 0, valor rec
  → Rechazo semántico verificado con éxito por el Servidor.
```

4.2. Tabla Consolidada de Mediciones Experimentales

Actividad	Servicio	Formato	Bytes Solicitud	Bytes Respuesta	Bytes Totales	Latencia RTT (ms)	Serialización (ms)	Validación Req (ms)	Deserialización (ms)	Sobrecarga Total (ms)
Actividad 1 (Con Schema)	`get_global_model`	**JSON**	86 B	146 B	**232 B**	4.339 ms	0.031 ms	0.638 ms	0.021 ms	**1.226 ms**
Actividad 1 (Con Schema)	`get_global_model`	**XML**	160 B	235 B	**395 B**	0.861 ms	0.124 ms	0.074 ms	0.096 ms	**0.708 ms**
Actividad 2 (Sin Schema / Código)	`training_report`	**JSON**	124 B	145 B	**269 B**	0.379 ms	0.015 ms	0.005 ms	0.012 ms	**0.033 ms**

5. Discusión y Análisis Comparativo

5.1. Comparación de Eficiencia de Datos: JSON vs. XML

- **Tamaño de Solicitud:** En la Actividad 1, la solicitud JSON consumió **86 bytes**, mientras que la versión equivalente en XML requirió **160 bytes**, lo que representa un incremento del **+86.05%** en XML.
- **Tamaño de Respuesta:** La respuesta JSON requirió **146 bytes** frente a **235 bytes** en XML (**+60.96%** más en XML).

- **Causa Estructural:** XML es intrínsecamente más verboso debido a la necesidad de etiquetas de apertura y cierre explícitas (`<client_id>client_1</client_id>` vs. `"client_id":"client_1"`), además del prólogo XML (`<?xml version="1.0" encoding="UTF-8"?>`). En sistemas distribuidos con miles de transacciones por segundo o clientes conectados por enlaces IoT limitados (como el nodo Raspberry Pi), JSON ofrece un ahorro considerable de ancho de banda.

5.2. Comparación de Sobrecarga de Serialización y Deserialización

- **Serialización:** La serialización a JSON tomó **0.031 ms** frente a **0.124 ms** en XML (~4 veces más rápido en JSON).
- **Deserialización:** La deserialización de JSON tardó **0.021 ms** frente a **0.096 ms** en XML (~4.5 veces más rápido en JSON).
- **Causa:** El motor `json` de Python está implementado en C puro y realiza mapeo directo a diccionarios. El parser XML (`xml.etree` / `lxml`) debe construir el árbol DOM de elementos y navegar nodos antes de extraer valores tipados.

5.3. Comparación de Validación: Esquema Formal vs. Validación en Código

- **Validación con JSON Schema:** Tomó **0.638 ms**, ya que la librería `jsonschema` en Python evalúa reglas complejas del metamodelo recursivamente en memoria.
- **Validación con XML XSD:** Tomó **0.074 ms**, beneficiándose de que el validador XSD de `lxml` compila el esquema contra la biblioteca nativa en C `libxml2` .
- **Validación Programática en Código (Actividad 2):** Tomó apenas **0.005 ms**.
  - **Ventaja de la validación en código:** Es extraordinariamente rápida (más de **100 veces más veloz** que la validación basada en esquemas genéricos en Python), consume memoria despreciable y permite validar relaciones dinámicas o cálculos entre campos que los esquemas declarativos estáticos no pueden expresar fácilmente.
  - **Desventaja de la validación en código:** Acopla estrechamente al cliente y al servidor en una tecnología o lenguaje específico, carece de un contrato formal legible por humanos o herramientas de autogeneración de SDKs, y traslada toda la responsabilidad del control de errores al programador.

---

6. Conclusiones

1. **Cumplimiento del Taller:** Se implementó con éxito la comunicación cliente-servidor distribuida para dos componentes físicos mediante sockets TCP con delimitación de tramas de 4 bytes, abordando tanto la validación estricta por esquemas predefinidos (JSON Schema y XSD) como la validación programática en código.
2. **Robustez ante Errores:** Tanto en la Actividad 1 como en la Actividad 2, los mecanismos de validación rechazaron de forma inmediata las solicitudes corruptas (e.g., valores negativos en campos restringidos o identificadores fuera de catálogo), evitando estados inconsistentes en el servidor.
3. **Decisión de Diseño para el Proyecto:** Para el proyecto final *AIR_QUALITY_DISTRIBUTED*, el formato **JSON** resulta superior para la transmisión periódica de pesos y deltas de red neuronal gracias a su menor huella de bytes y su baja latencia de serialización, mientras que los contratos **JSON Schema** garantizan una interfaz formal y segura entre nodos heterogéneos.

---

7. Instrucciones para Ejecutar entre Dos Máquinas

Para replicar este taller en dos máquinas físicas conectadas a la misma red:

En la Máquina 1 (Servidor Coordinador):

```
# Conocer la IP local (por ejemplo 192.168.1.50)
python workshop7_server.py --host 0.0.0.0 --port 5000
```

En la Máquina 2 (Cliente PC o Raspberry Pi):

```
# Reemplazar con la IP del Servidor Coordinador
python workshop7_client.py --server-host 192.168.1.50 --port 5000 --client-id client_1
```